Py-ChemShell 25.0.6 installation on HIGGS with DL_POLY 5 and ORCA 6.1.0

A step-by-step guide for the philomath curious about QM/MM

`Version 0.98.1`

William Houppy

`https://github.com/biochemix/pychemshell-dlpoly-orca`

20th July 2026

Abstract

This guide walks you through installing **Py-ChemShell 25.0.6** in your own account on the HIGGS high-performance computing (HPC) cluster, from the source *tarball*, and using it to run a hybrid quantum-mechanics/molecular-mechanics (QM/MM) calculation. The quantum region is handled by **ORCA** and the molecular-mechanics region by **DL_POLY 5**. Every command is shown explicitly, together with the output you should expect, so that you can follow along even if this is your first time using an HPC cluster. No administrator (*root*) rights are required: everything is built in your home directory and loaded through the *environment modules* system.

Contents

0 Background: what are we installing, and why? 2
0.0 The HIGGS cluster in one paragraph 2

1 Obtaining and unpacking the tarball 3

2 Loading and checking the environment modules 3
2.0 Start from a clean environment 3
2.1 Add a suitable Python 3
2.2 Locate ORCA 4
2.3 Check the compilers and library paths 4

3 Patching the installer (one small fix) 4

4 Configuring and building Py-ChemShell 5

5 Verifying the build 6

6 A first QM/MM test (serial) 6
6.0 Set up a test directory ... 6
6.1 Write the ChemShell input ... 6
6.2 Write the SLURM job script ... 7
6.3 Submit and check ... 8

7 Setting up Py-ChemShell with parallel ORCA (butanol-1 example) 8
7.0 Why a wrapper script is needed ... 8
7.1 Tell ChemShell to use the wrapper ... 9
7.2 One setting for the core count: passing it via SLURM ... 9
7.3 Submit and check ... 10
7.4 Recap: the five files for the parallel butanol-1 example ... 11

8 Summary 11

A Troubleshooting 12

0 Background: what are we installing, and why?

Many interesting chemical problems, for example how an enzyme catalyses a reaction, involve a small, chemically active region embedded in a very large environment. Treating *everything* with quantum mechanics is far too expensive. The **QM/MM** approach solves this by describing only the reactive part with quantum mechanics (QM) and the surrounding bulk with cheaper molecular-mechanics (MM) force fields.

ChemShell is the "glue" that couples a QM program to an MM program and drives the calculation (energies, geometry optimisations, and so on). In this guide we use three pieces of software:

- **Py-ChemShell 25.0.6**: the Python version of ChemShell, the driver that orchestrates the QM/MM calculation.
- **ORCA 6.1.0**: the QM engine (already installed on HIGGS by the administrators; we simply call it).
- **DL_POLY 5**: the MM engine, downloaded and compiled automatically by the ChemShell installer.

0.0 The HIGGS cluster in one paragraph

HIGGS is managed with **EasyBuild** and the **environment modules** system. Software is not installed system-wide; instead you *load* the pieces you need for a session with the `module load` command, and they disappear again when you log out or run `module purge`. Jobs are submitted to compute nodes through the **SLURM** scheduler. The compute nodes each have 192 physical CPU cores (two sockets of 96 cores, no hyper-threading). A short `cpudebug` partition (maximum one hour) is available for quick tests, and we use it throughout this guide.

Note. Throughout, `$HOME` means your home directory (e.g. `/hpcfs/home/<your-username>`) and `$` at the start of a line is the shell prompt (you do *not* type it).

1 Obtaining and unpacking the tarball

Py-ChemShell is freely available and can be downloaded from its website (`https://www.chemshell.org`) as a tarball. A *tarball* is a compressed archive (extension `.tar.gz`) containing the ChemShell source code. Place it in a dedicated folder in your home directory. Here we assume it is already in `$HOME/chemshell/`.

```
# 1. Move into a working folder for ChemShell
cd $HOME/chemshell

# 2. Check the tarball is there (about 35 MB)
ls -lh chemsh-py-25.0.6.tar.gz

# 3. Unpack it
tar xzf chemsh-py-25.0.6.tar.gz

# 4. Enter the source directory that was created
cd chemsh-py-25.0.6
ls
```

You should see the top-level contents of the distribution:

```
AUTHORS  chemsh  COPYING  COPYING.LESSER  docs  INSTALL  README.md  setup  tests
```

The `setup` script is the installer; the `tests` folder contains ready-made examples that we will reuse later.

Note. Always unpack into a *clean* directory. If you have to start over, delete the `chemsh-py-25.0.6` folder and unpack the tarball again, rather than building on top of a half-finished attempt.

2 Loading and checking the environment modules

Before building anything we choose a consistent set of compilers and libraries. We use the `foss/2024a` toolchain (Free and Open-Source Software): GCC 13.3.0, OpenMPI 5.0.3, and the OpenBLAS/FlexiBLAS, FFTW and ScaLAPACK maths libraries. ChemShell also needs Python $\geq$ 3.10, which is a separate module.

2.0 Start from a clean environment

```
module purge
module load foss/2024a
module list
```

`module list` should show the toolchain components, including `GCC/13.3.0`, `OpenMPI/5.0.3`, `OpenBLAS/0.3.27`, `FlexiBLAS/3.4.4`, `FFTW/3.3.10` and `ScaLAPACK/2.2.0-fb`.

2.1 Add a suitable Python

The Python built for the *same* compiler core (`GCCcore-13.3.0`) keeps everything consistent:

```
module load Python/3.12.3-GCCcore-13.3.0
which python3
python3 --version        # must report 3.12.3, NOT the system 3.9
```

Note. Loading Python may print an *Lmod* warning that a few small helper modules (`XZ`, `binutils`, `zlib`) were reloaded to their `GCCcore-13.3.0` versions. This is harmless: the module system is simply keeping the compiler chain consistent.

2.2 Locate ORCA

ORCA is pre-installed. Find its module and the full path to the executable:

```
module avail orca
# Expected: ORCA/6.1.0-gompi-2023b-avx2
```

The important detail for later: this ORCA was built with a *different* toolchain (`gompi-2023b`, i.e. OpenMPI 4.1.6), not the OpenMPI 5.0.3 we build ChemShell against. We will handle that carefully in Section 7. For now, simply record the executable's absolute path:

```
module load ORCA/6.1.0-gompi-2023b-avx2
which orca
# /hpcfs/software/EasyBuild/easybuild/software/ORCA/6.1.0-gompi-2023b-avx2/bin/orca
module unload ORCA/6.1.0-gompi-2023b-avx2   # unload again before building ChemShell
```

2.3 Check the compilers and library paths

EasyBuild sets convenient variables (`$EBROOT...`) pointing at each library. Confirm the MPI compiler wrappers and the maths libraries resolve:

```
which mpif90 mpicc mpicxx
echo $EBROOTFLEXIBLAS
echo $EBROOTFFTW
echo $EBROOTSCALAPACK
ls $EBROOTFLEXIBLAS/lib*/libflexiblas.so
```

The `mpif90`/`mpicc`/`mpicxx` wrappers should live under `OpenMPI/5.0.3-GCC-13.3.0/bin`, and `libflexiblas.so` should exist (it is present under both `lib` and `lib64`; we use the `lib` copy).

3 Patching the installer (one small fix)

The installer's default compilers are Intel (`ifort`/`icx`). We are using GCC, and we must also enable MPI. On this version there is a small bug in the installer: when MPI is switched on, one line looks up a dictionary with a text key instead of a true/false value, which crashes the script with a `KeyError`.

The fix is a one-character-level change: wrap the lookup in `bool(...)`. A robust way to apply it (independent of the exact line number) is:

```
# Show the offending line first (optional)
grep -n "bools\[args.mpi\]" setup

# Apply the fix
sed -i "s/bools\[args.mpi\]/bools[bool(args.mpi)]/" setup

# Confirm it is now bools[bool(args.mpi)]
grep -n "bools\[bool(args.mpi)\]" setup
```

Note. This is a genuine upstream bug and should be reported to the ChemShell developers. It does not affect the correctness of your results: it only prevents the installer from running until patched.

4 Configuring and building Py-ChemShell

Now build ChemShell together with DL_POLY 5. The installer downloads DL_POLY from its official repository and compiles it automatically. ORCA is *not* built here. It is an external program we call at run time, so it needs no `--orca` flag.

Make sure `foss/2024a` and the Python module are loaded (and ORCA is *not* loaded), then run:

```
./setup \
  -fc mpif90 -cc mpicc -cpc mpicxx \
  -mpi=openmpi \
  --python_root_dir $EBROOTPYTHON \
  --blas $EBROOTFLEXIBLAS/lib/libflexiblas.so \
  --lapack $EBROOTFLEXIBLAS/lib/libflexiblas.so \
  --fftw $EBROOTFFTW \
  --scalapack \
  --dl_poly \
  -j 4
```

What each part means:

- `-fc/-cc/-cpc` — the Fortran, C and C++ MPI compiler wrappers.
- `-mpi=openmpi` — build with MPI parallel support.
- `--python_root_dir $EBROOTPYTHON` — forces the correct Python 3.12, not the old system Python.
- `--blas / --lapack` — point at FlexiBLAS (our maths library).
- `--fftw / --scalapack` — fast Fourier transforms and scalable linear algebra.
- `--dl_poly` — download and build the DL_POLY 5 MM engine.
- `-j 4` — compile using 4 parallel jobs (faster).

The build takes a couple of minutes. Some warnings scroll past (for example type-mismatch warnings in `dlf_mpi.f90`, an `executable stack` note, and two `not an accessible directory` warnings about the BLAS/LAPACK paths). These are all **harmless**. The build has finished successfully when you see progress reach 100% and a line similar to:

```
[100%] Built target namd

 >>> Time used for building: 109.447 s
```

5 Verifying the build

Check that the three key products exist and that the driver runs:

```
ls -lh bin/gnu/chemsh.x bin/gnu/chemsh
ls -lh build/gnu/lib/libdl_poly.so
./bin/gnu/chemsh -v
```

The last command should print:

```
chemsh: ChemShell version 25.0
```

If you see the version string, Py-ChemShell is installed. `chemsh` is a small launcher script; `chemsh.x` is the compiled program; and `libdl_poly.so` is the DL_POLY interface library that ChemShell calls in-process.

6 A first QM/MM test (serial)

We start with a small, *serial* (single-core) test to confirm the whole chain (ChemShell → ORCA → DL_POLY) works. Rather than invent a system, we reuse a *validated* example shipped in the `tests` folder (an *n*-butanol molecule) and simply swap the QM engine to ORCA. Because the example has a known reference energy, a correct result is easy to recognise.

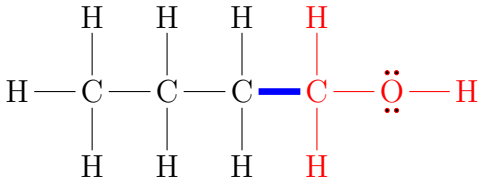


Figure 0: QM/MM partition of butanol-1.
The atoms and bonds are drawn in **red** in the **QM region** (`qm_region=range(5)`): the carbinol carbon, its two hydrogens and the hydroxyl O–H, treated by ORCA. The propyl tail in **black** is the **MM region**, treated by DL_POLY. The thick **blue** bond joining the two carbons is the **QM/MM boundary in mechanical embedding** in which the interaction across the cut is handled entirely by classical force field.

6.0 Set up a test directory

```
mkdir -p $HOME/chemshell/qmmm_test
cd $HOME/chemshell/qmmm_test

# Copy the shipped geometry and force field
cp $HOME/chemshell/chemsh-py-25.0.6/tests/qmmm/butanol.cjson .
cp $HOME/chemshell/chemsh-py-25.0.6/tests/qmmm/nwchem-dl_poly/butanol.ff .
```

6.1 Write the ChemShell input

Create the file `butanol_orca_dlpoly.py` by pasting this whole block into the terminal:

```
cat > butanol_orca_dlpoly.py << 'EOF'
from chemsh import *

# Load the molecule (geometry supplied with ChemShell)
frag = Fragment(coords='butanol.cjson')

# QM engine: ORCA, Hartree-Fock with a small basis, run on 1 core
qm = ORCA(method="hf", basis="6-31g", nprocs=1)

# MM engine: DL_POLY, using the shipped force field
mm = DL_POLY(ff='butanol.ff', rcut=100.0)

# Couple them: first 5 atoms are the QM region, mechanical embedding
qmmm = QMMM(frag=frag, qm_region=range(5), qm=qm, mm=mm, embedding='mechanical')

# One geometry optimisation
opt = Opt(theory=qmmm, algorithm="lbfgs", tolerance=0.0003,
          maxcycle=300, maxene=400, trust_radius='energy')
opt.run()

print("QM/MM optimised energy =", opt.result.energy, "Hartree")
EOF
```

6.2 Write the SLURM job script

Because ORCA is run on a single core here, there is no MPI to worry about yet: we only need ORCA on the `PATH` and its own library directory on `LD_LIBRARY_PATH`. Create `run_qmmm.slurm` by pasting this block into the terminal:

```
cat > run_qmmm.slurm << 'EOF'
#!/bin/bash --login
#SBATCH --job-name=qmmm_smoke
#SBATCH --partition=cpudebug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --time=00:20:00
#SBATCH --output=qmmm_smoke.%j.out

# ChemShell build environment
module purge
module load foss/2024a Python/3.12.3-GCCcore-13.3.0

# Point at ORCA (executable + its own libraries). Do NOT module-load ORCA here.
ORCADIR=/hpcfs/software/EasyBuild/easybuild/software/ORCA/6.1.0-gompi-2023b-avx2
export PATH=$ORCADIR/bin:$PATH
export LD_LIBRARY_PATH=$ORCADIR/lib:$LD_LIBRARY_PATH

CHEMSH=$HOME/chemshell/chemsh-py-25.0.6/bin/gnu/chemsh

cd $SLURM_SUBMIT_DIR
$CHEMSH butanol_orca_dlpoly.py > butanol_orca_dlpoly.log 2>&1
echo "chemsh exit=$?"
EOF
```

6.3 Submit and check

```
sbatch run_qmmm.slurm
squeue -u $USER            # watch it run (a minute or so)
```

When it finishes, inspect the results:

```
cat qmmm_smoke.*.out
tail -5 butanol_orca_dlpoly.log
```

A successful run prints `chemsh exit=0` and an energy very close to the ORCA reference value of −114.99042281 Hartree, for example:

```
=== QM/MM optimised energy = [-114.99042314] Hartree ===
 ChemShell exiting normally.
```

Agreement to within about 3×10^{-7} Hartree confirms the full chain works.

7 Setting up Py-ChemShell with parallel ORCA (butanol-1 example)

In real QM/MM work, the QM calculation dominates the run time (often around 95%), so we want ORCA to use many CPU cores. This section works through a complete parallel example on the same butanol-1 system used in Section 6, so you can compare the two directly. It introduces one subtlety that must be handled correctly, then gives every file you need.

7.0 Why a wrapper script is needed

ChemShell was built against **OpenMPI 5.0.3**, but ORCA was built against **OpenMPI 4.1.6**. Both provide a library with the *same* file name (`libmpi.so.40`). If both versions were visible at once, ChemShell could accidentally pick up ORCA's 4.1.6 and misbehave in ways that are hard to diagnose.

The clean solution is to give each program its own private environment. We keep ChemShell in the `foss/2024a` environment, and we launch ORCA through a small **wrapper script** that loads ORCA's own modules *inside a separate subprocess*. The two never see each other's MPI.

Create `orca_parallel.sh` in the test directory and make it executable by pasting this block into the terminal:

```
cat > orca_parallel.sh << 'EOF'
#!/bin/bash --login
# Isolated launcher: ORCA gets its OWN OpenMPI 4.1.6 environment,
# separate from the ChemShell process that called it.
module purge
module load ORCA/6.1.0-gompi-2023b-avx2

# ORCA needs its full absolute path to spawn MPI workers correctly:
exec /hpcfs/software/EasyBuild/easybuild/software/ORCA/6.1.0-gompi-2023b-avx2/bin/
    ↪ orca "$@"
EOF
```

```
chmod +x orca_parallel.sh
```

7.1 Tell ChemShell to use the wrapper

Create a parallel version of the input, `butanol_orca_dlpoly_par.py`. The only changes from the serial version are (i) `path` points at the wrapper, and (ii) the number of ORCA ranks is read from SLURM automatically (see Section 7.2). Paste this block into the terminal:

```
cat > butanol_orca_dlpoly_par.py << 'EOF'
import os
from chemsh import *

# Number of ORCA MPI ranks, taken from the SLURM job (falls back to 1)
norca = int(os.environ.get('SLURM_NTASKS', 1))

frag = Fragment(coords='butanol.cjson')

qm = ORCA(method="hf", basis="6-31g",
          path="./orca_parallel.sh",    # launch ORCA through the wrapper
          nprocs=norca)                 # run ORCA in parallel

mm = DL_POLY(ff='butanol.ff', rcut=100.0)

qmmm = QMMM(frag=frag, qm_region=range(5), qm=qm, mm=mm,
            embedding='mechanical')

opt = Opt(theory=qmmm, algorithm="lbfgs", tolerance=0.0003,
          maxcycle=300, maxene=400, trust_radius='energy')
opt.run()

print("QM/MM optimised energy =", opt.result.energy, "Hartree")
EOF
```

Note. In this setup ChemShell runs as a single process and calls DL_POLY in-process (through `libdl_poly.so`), while only ORCA runs in parallel. The QM and MM steps happen one after another, not at the same time, so you do *not* need to divide cores between ORCA and DL_POLY: you simply size the job for ORCA.

7.2 One setting for the core count: passing it via SLURM

It is easy to make mistakes if the number of cores is typed in two places (the job script and the ChemShell input). We avoid this by setting it **once** in the SLURM script. SLURM automatically exports `--ntasks` as the environment variable `$SLURM_NTASKS`, and our Python input reads that variable (the `os.environ.get('SLURM_NTASKS', 1)` line above).

Create the parallel job script `run_qmmm_par.slurm` by pasting this block into the terminal:

```
cat > run_qmmm_par.slurm << 'EOF'
#!/bin/bash --login
#SBATCH --job-name=qmmm_par
#SBATCH --partition=cpudebug
#SBATCH --nodes=1
#SBATCH --ntasks=4               # <-- the ONLY place the core count is set
#SBATCH --time=00:20:00
#SBATCH --output=qmmm_par.%j.out

# ChemShell environment only (OpenMPI 5.0.3). ORCA's 4.1.6 is loaded
# separately, inside orca_parallel.sh.
module purge
module load foss/2024a Python/3.12.3-GCCcore-13.3.0

CHEMSH=$HOME/chemshell/chemsh-py-25.0.6/bin/gnu/chemsh

echo "ORCA ranks (SLURM_NTASKS) = $SLURM_NTASKS"
cd $SLURM_SUBMIT_DIR
$CHEMSH butanol_orca_dlpoly_par.py > butanol_orca_dlpoly_par.log 2>&1
echo "chemsh exit=$?"
EOF
```

To change the number of cores later, edit only the `--ntasks` line and the value flows automatically into ORCA. Because `--nodes=1` places all the tasks on a single node, `--ntasks=4` already reserves the four "slots" on that node for ORCA's internal `mpirun`, so no `--ntasks-per-node` line is needed.

7.3 Submit and check

```
sbatch run_qmmm_par.slurm
squeue -u $USER
```

After it finishes:

```
cat qmmm_par.*.out
tail -5 butanol_orca_dlpoly_par.log
```

You should again get `chemsh exit=0` and the same energy (≈ -114.99042 Hartree) as the serial run: the correct answer, now with ORCA running on several cores.

Note. Parallel run is far slower than serial on a tiny test. This is expected for very small systems: parallel overhead dominates. Benchmark on a realistic-sized molecule instead.

7.4 Recap: the five files for the parallel butanol-1 example

For reference, a complete parallel QM/MM run of butanol-1 uses these files, all in `$HOME/chemshell/qmmm_test`:

- `butanol.cjson`: the geometry, copied from the shipped tests (Section 6).
- `butanol.ff`: the force field, copied from the shipped tests (Section 6).
- `orca_parallel.sh`: the wrapper that gives ORCA its own MPI environment.
- `butanol_orca_dlpoly_par.py`: the ChemShell input, which calls ORCA through the wrapper and reads the rank count from `$SLURM_NTASKS`.
- `run_qmmm_par.slurm`: the job script, where `--ntasks` is the single place you choose how many cores ORCA uses.

To run more cores, change only `--ntasks` then restart with `sbatch run_qmmm_par.slurm`.

8 Summary

You have:

1. unpacked the Py-ChemShell 25.0.6 tarball into your home directory;
2. loaded a consistent open-source toolchain (`foss/2024a`) plus a suitable Python;
3. applied the one required fix to the installer;
4. built Py-ChemShell together with DL_POLY 5;
5. verified the build and ran a validated serial QM/MM test with ORCA;
6. set up parallel ORCA cleanly using an isolating wrapper script; and
7. driven the core count from a single SLURM setting.

Your installation uses only open-source components for ChemShell and DL_POLY, and calls the pre-installed ORCA as an external program: a robust, reproducible setup for QM/MM chemistry on HIGGS.

It does not yet exercise electrostatic embedding, link-atom QM/MM boundaries, or periodic MM electrostatics (Ewald/PME), which a protein/water run requires. These are features of the installed stack rather than new software, but remain untested in this build. A small electrostatic-embedding and link-atom test would confirm those code paths before production enzyme calculations. Actual larger QM/MM experiments would have to be carefully configured and tuned with the optimal number of CPU cores.

A Troubleshooting

- **The installer stops with `KeyError:'openmpi'` during `./setup`.**
  The installer was not patched. Apply the `sed` fix in Section 3 and run `./setup` again.
- **CMake reports `Could NOT find Python3` or a `Wrong version` message.**
  The Python module was not loaded, so the installer found the old system Python 3.9. Load `Python/3.12.3-GCCcore-13.3.0` and pass `--python_root_dir $EBROOTPYTHON` to `./setup`.
- **The parallel ORCA job aborts during `Startup` with a `not enough slots` message.**
  SLURM did not reserve enough slots. With `--nodes=1`, make sure `--ntasks` equals the number of ORCA ranks you asked for (the `nprocs` value ORCA uses).
- **You want to start the build again from scratch.**
  Run `./setup --clean` in the source directory, or delete the `chemsh-py-25.0.6` folder and re-unpack the tarball.